Johns Hopkins Engineering

Introduction to GPU Programming

Module 8 Lecture 8C NVidia GPU-Accelerated Libraries – Linear Algebra Libraries

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Chancellor T. Pascale
Computer Science Department

Overview

6. cuBLAS
7. cuSPARSE
8. cuSOLVER

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.6 cuBLAS

derived from https://developer.nvidia.com/cublas

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

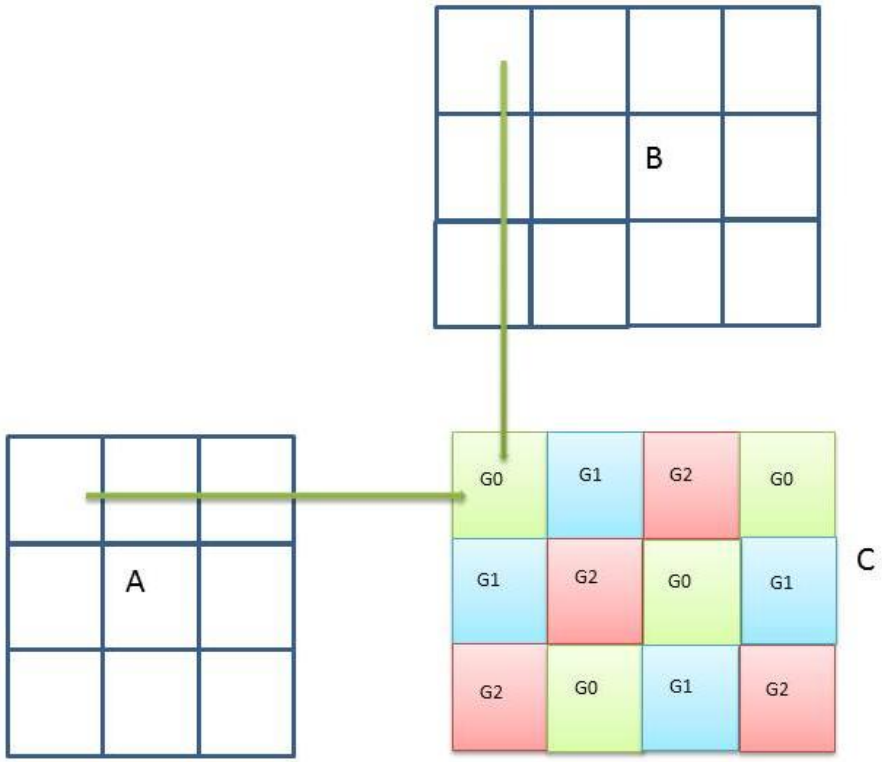
8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Intro

- The NVIDIA cuBLAS library is a fast GPU-accelerated implementation of the standard basic linear algebra subroutines (BLAS). Using cuBLAS APIs, you can speed up your applications by deploying compute-intensive operations to a single GPU or scale up and distribute work across multi-GPU configurations efficiently.
- NVBLAS is a GPU-accelerated version of BLAS that further accelerates BLAS Level-3 routines by dynamically routing BLAS calls to one or more NVIDIA GPUs as well as CPUs in the system through the cuBLAS-XT interface.
- Researchers and scientists use cuBLAS for developing gpu-accelerated algorithms in areas including high performance computing, image analysis and machine learning.

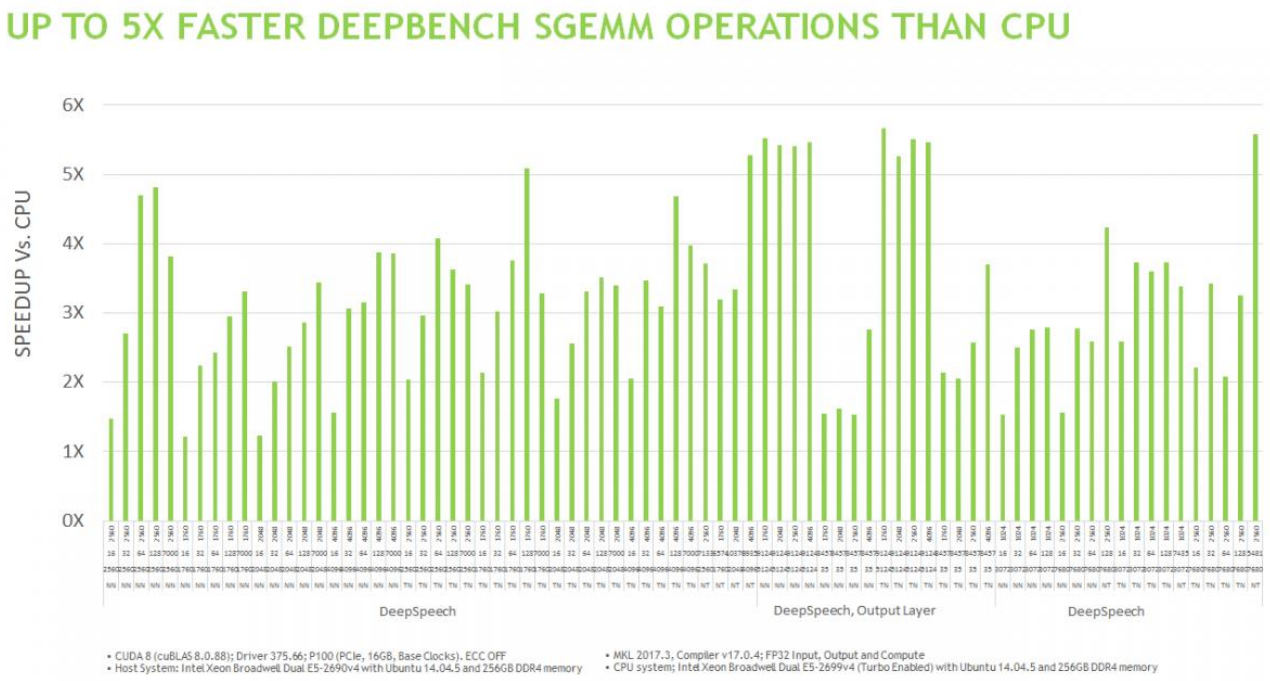
8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Intro



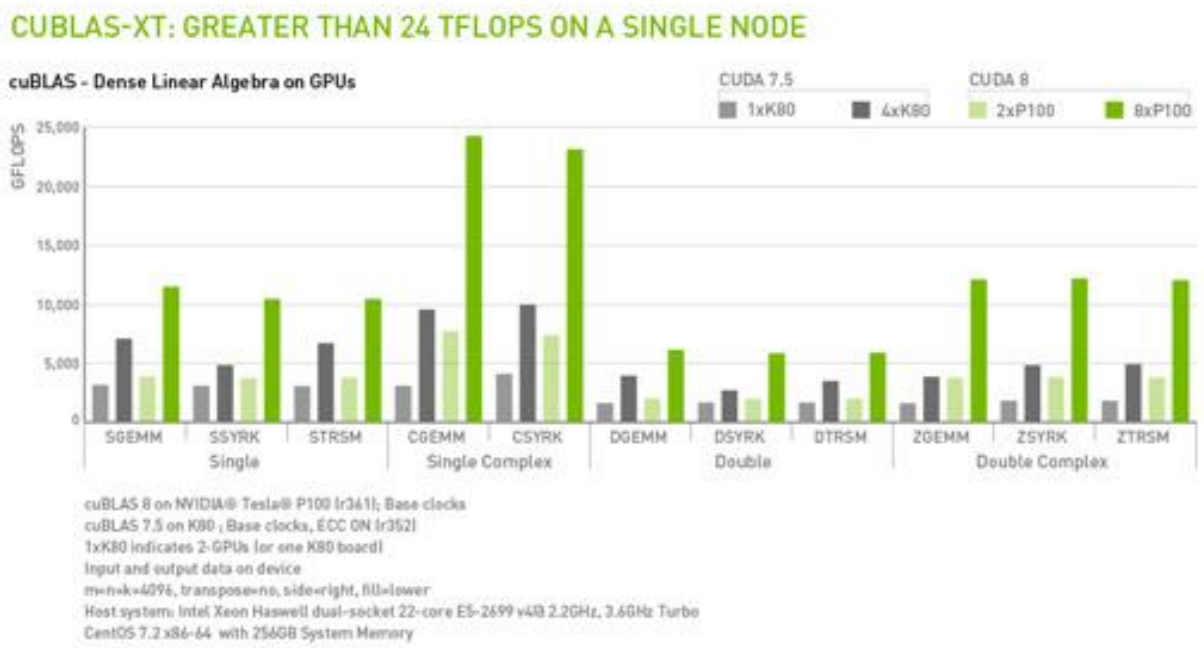
8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Performance



8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Scalability



8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Key Features

- Complete support for all 152 standard BLAS routines
- Single, double, complex, and double complex data types
- Supports half-precision (FP16) and integer (INT8) matrix multiplication operations
- Support for multiple GPUs and concurrent kernels
- Supports CUDA streams for concurrent operations
- Fortran bindings
- Batch processing APIs for high performance GEMM operations, LU factorization, and matrix inverse operations
- Device API that can be called from with your own CUDA kernels
- New implementation of TRSV (Triangular solve), up to 7x faster than previous implementation

8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Sample Code

```
__global__ void invokeDeviceCublasSgemm(cublasStatus_t *returnValue,
     int n, const float *d_alpha,
     const float *d_A, const float *d_B,
     const float *d_beta, float *d_C)
{
     cublasHandle_t cnpHandle;
     cublasStatus_t status = cublasCreate(&cnpHandle);
     if (status != CUBLAS_STATUS_SUCCESS) {
          *returnValue = status;
          return;
     }
      /* Perform operation using cublas */
     status = cublasSgemm(cnpHandle, CUBLAS_OP_N,
          CUBLAS_OP_N, n, n, n,d_alpha, d_A, n, d_B,
          n, d_beta, d_C, n);
     cublasDestroy(cnpHandle);
     *returnValue = status;
}
```

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.7 cuSPARSE

derived from https://developer.nvidia.com/cusparse

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

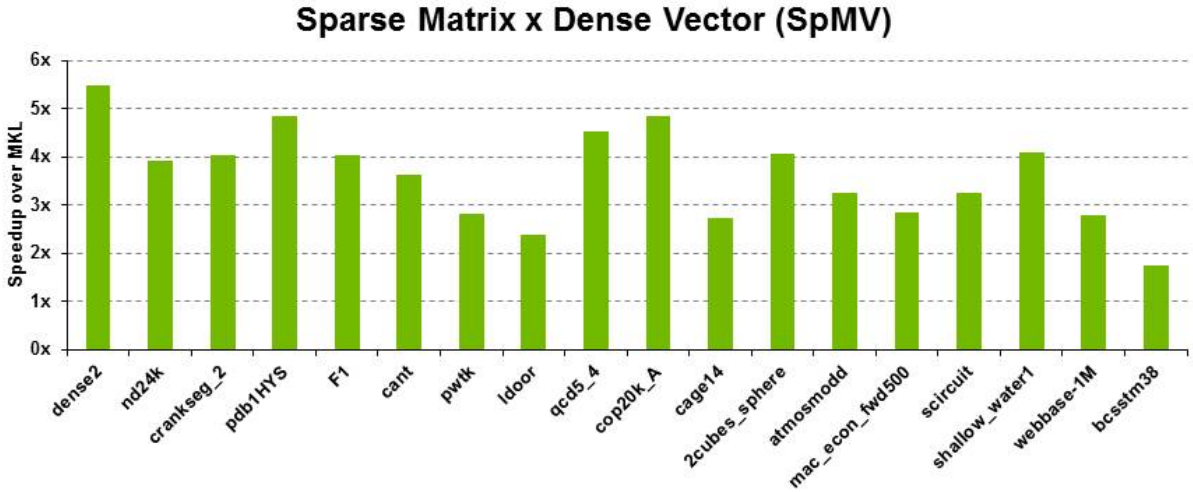
Intro

- The NVIDIA CUDA Sparse Matrix library (cuSPARSE) provides a collection of basic linear algebra subroutines used for sparse matrices that delivers up to 8x faster performance than the latest MKL. The cuSPARSE library is designed to be called from C or C++, and the latest release includes a sparse triangular solver.

8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Performance

cuSPARSE: 5x Faster than MKL



- Average of s/c/d/z routines
- cuSPARSE 6.0 on K40m, ECC ON, input and output data on device
- MKL 11.0.4 on Intel IvyBridge 12-core E5-2697 v2 @ 2.70GHz
- Matrices obtained from: http://www.cise.ufl.edu/research/sparse/matrices/

Performance may vary based on OS version and motherboard configuration

8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Key Features

- Supports dense, COO, CSR, CSC, ELL/HYB and Blocked CSR sparse matrix formats
- Level 1 routines for sparse vector x dense vector operations
- Level 2 routines for sparse matrix x dense vector operations
- Level 3 routines for sparse matrix x multiple dense vectors (tall matrix)
- Routines for sparse matrix by sparse matrix addition and multiplication
- Conversion routines that allow conversion between different matrix formats
- Sparse Triangular Solve
- Tri-diagonal solver
- Incomplete factorization preconditioners ilu0 and ic0

8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Sample Code

```
//step 1: transform CSR to BSR with column-major order
int base, nnz;
cusparseDirection_t dirA = CUSPARSE_DIRECTION_COLUMN;
int mb = (m + blockDim-1)/blockDim; int nb = (n + blockDim-1)/blockDim;
cudaMalloc((void**)&bsrRowPtrC, sizeof(int) *(mb+1));
cusparseXcsr2bsrNnz(handle, dirA, m, n, descrA, csrRowPtrA, csrColIndA,
                                        blockDim, descrC, bsrRowPtrC);
cudaMemcpy(&nnzb, bsrRowPtrC+mb, sizeof(int), cudaMemcpyDeviceToHost);
cudaMemcpy(&base, bsrRowPtrC , sizeof(int), cudaMemcpyDeviceToHost);
nnzb -= base;
cudaMalloc((void**)&bsrColIndC, sizeof(int)*nnzb);
cudaMalloc((void**)&bsrValC, sizeof(float)*(blockDim*blockDim)*nnzb);
cusparseScsr2bsr(handle, dirA, m, n, descrA, csrValA, csrRowPtrA,
     csrColIndA, blockDim, descrC, bsrValC, bsrRowPtrC, bsrColIndC);
// step 2: allocate vector x and vector y large enough for bsrmv
cudaMalloc((void**)&x, sizeof(float)*(nb*blockDim));
cudaMalloc((void**)&y, sizeof(float)*(mb*blockDim));
cudaMemcpy(x, hx, sizeof(float)*n, cudaMemcpyHostToDevice);
cudaMemcpy(y, hy, sizeof(float)*m, cudaMemcpyHostToDevice);
// step 3: perform bsrmv
cusparseSbsrmv(handle, dirA, transA, mb, nb, alpha, descrC, bsrValC,
bsrRowPtrC, bsrColIndC, blockDim, x, beta, y);
```

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

8.8 cuSOLVER

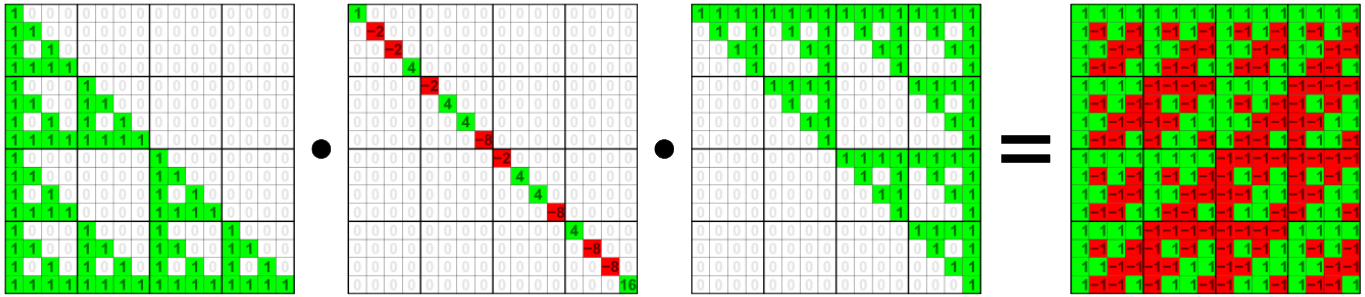
derived from https://developer.nvidia.com/cusolver

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Intro

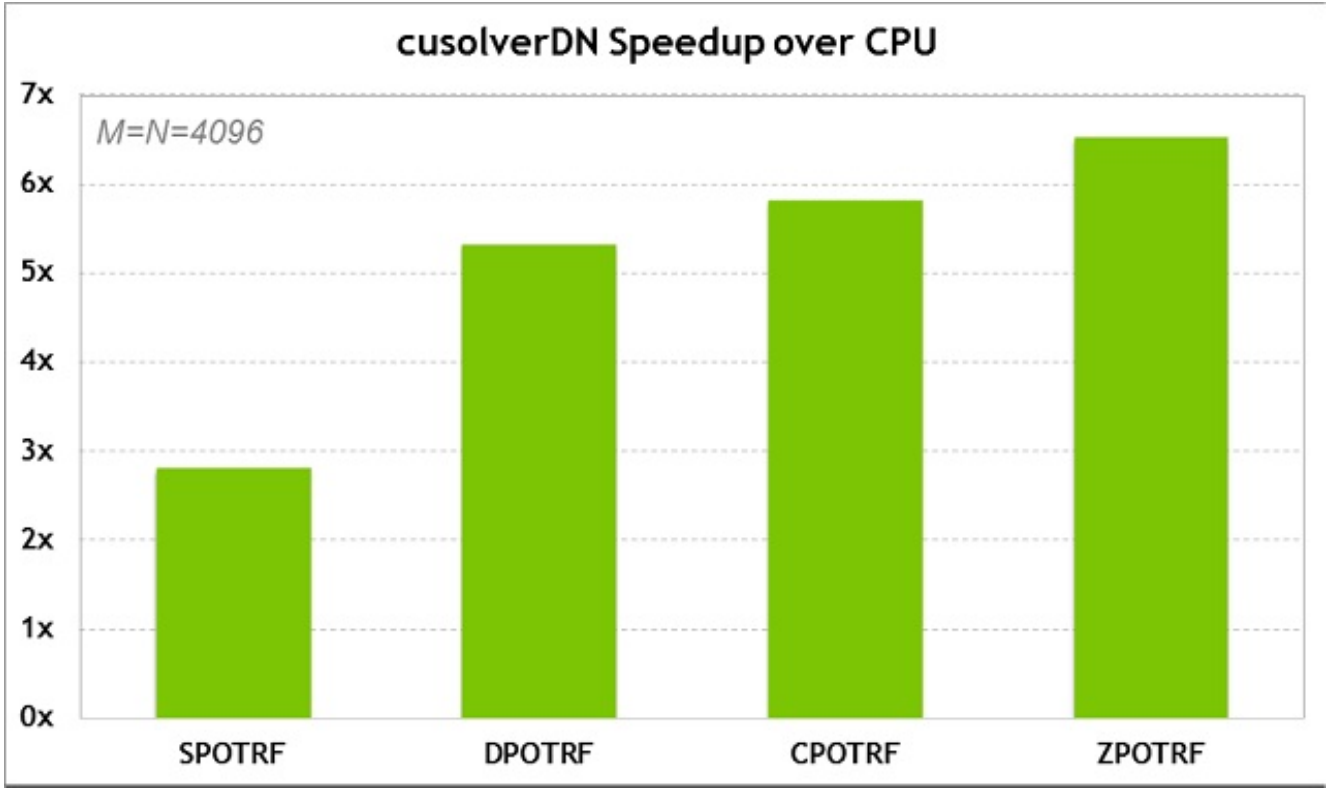
- The NVIDIA cuSOLVER library provides a collection of dense and sparse direct solvers which deliver significant acceleration for Computer Vision, CFD, Computational Chemistry, and Linear Optimization applications.



https://devblogs.nvidia.com/parallelforall/parallel-direct-solvers-with-cusolver-batched-qr/

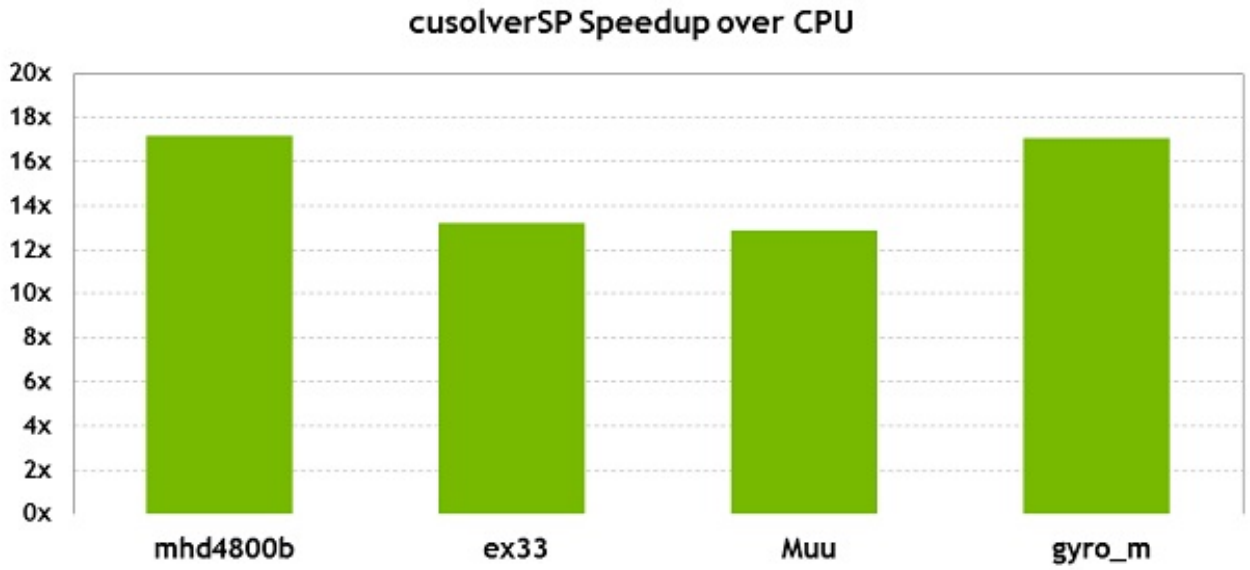
8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Performance



8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Performance



8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Key Features
8.8.4 Sample Code

Key Features

cusolverDN: Key LAPACK dense solvers 3-6x faster than MKL.

- Dense Cholesky, LU, SVD, QR
- Applications include: optimization, Computer Vision, CFD

cusolverSP

- Sparse direct solvers & Eigensolvers
- Applications include: Newton's method, Chemical Kinetics

cusolverRF

- Sparse refactorization solver
- Applications include: Chemistry, ODEs, Circuit simulation

8.6 cuBLAS
8.6.1 Intro
8.6.2 Performance
8.6.3 Scalability
8.6.4 Key Features
8.6.5 Sample Code
8.7 cuSPARSE
8.7.1 Intro
8.7.2 Performance
8.7.3 Key Features
8.7.4 Sample Code
8.8 cuSOLVER
8.8.1 Intro
8.8.2 Performance
8.8.3 Scalability
8.8.4 Key Features
8.8.5 Sample Code

Sample Code

```
cusolver_status = cusolverSpCreate(&cusolverH);

cusparse_status = cusparseCreateMatDescr(&descrA);

cusparseSetMatType(descrA, CUSPARSE_MATRIX_TYPE_GENERAL);

cusparseSetMatIndexBase(descrA, CUSPARSE_INDEX_BASE_ONE); // base-1

cusolver_status = cusolverSpCreateCsrqrInfo(&info);

cusolver_status = cusolverSpXcsrqrAnalysisBatched( cusolverH, m, m, nnzA,
                            descrA, d_csrRowPtrA, d_csrColIndA, info);

cusolver_status = cusolverSpDcsrqrBufferInfoBatched( cusolverH, m, m,
                        nnzA, descrA, d_csrValA, d_csrRowPtrA, d_csrColIndA,
                        batchSize, info, &size_internal, &size_qr);

cusolver_status = cusolverSpDcsrqrsvBatched( cusolverH, m, m, nnzA,
                        descrA, d_csrValA, d_csrRowPtrA, d_csrColIndA, d_b,
                        d_x, batchSize, info, buffer_qr);
```